Rent price prediction in Berlin

Data Visualizations and Insights

[1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import altair as alt

alt.data_transformers.disable_max_rows()
alt.renderers.enable("default")
```

[1]:
```
RendererRegistry.enable('default')
```

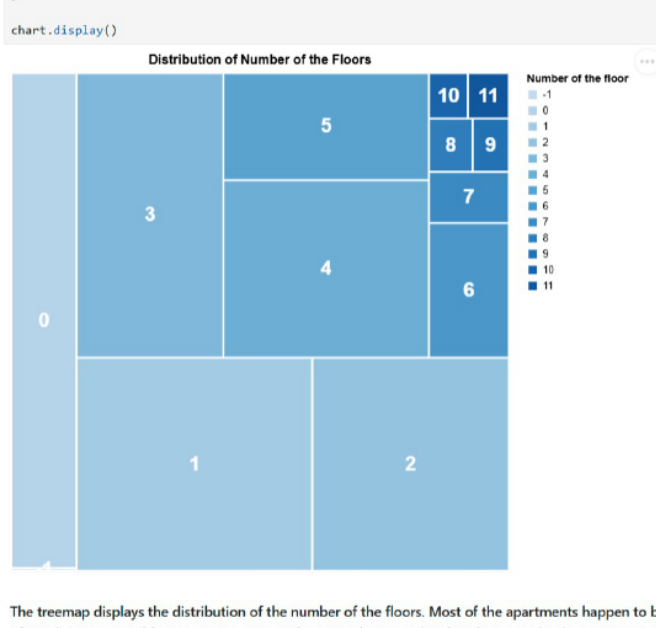
[2]:
```
df = pd.read_csv('../datasets/rent_listings_final.csv')
df.shape
```

[2]:
```
(15588, 23)
```

[3]:
```
df.info()
```

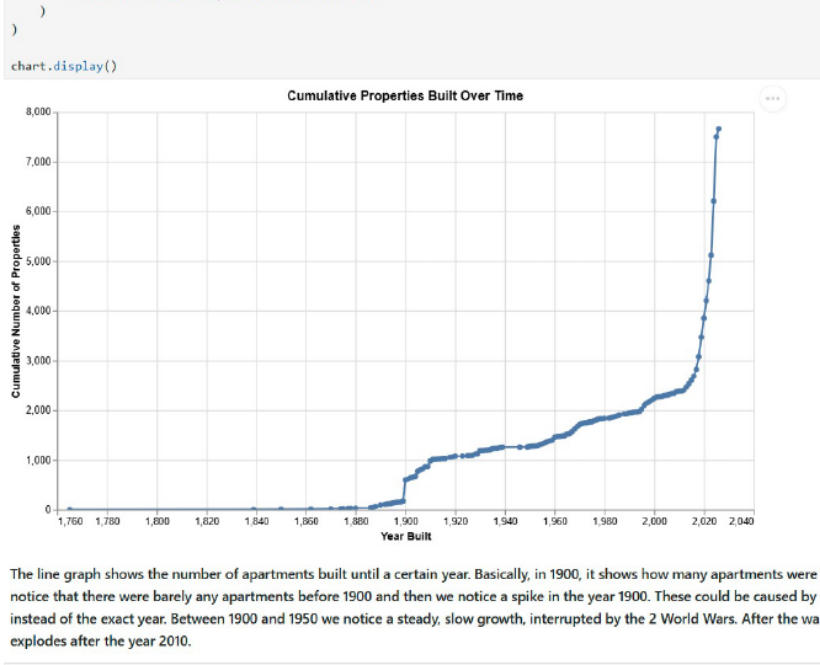
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15588 entries, 0 to 15587
Data columns (total 23 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   listing_url          15588 non-null  object
 1   cold_rent            15588 non-null  float64
 2   area                 15588 non-null  float64
 3   rooms                15588 non-null  float64
 4   floor                13646 non-null  float64
 5   free_from            15588 non-null  object
 6   has_balkon           15588 non-null  bool
 7   has_terrasse         15588 non-null  bool
 8   has_garten           15588 non-null  bool
 9   elevator             15588 non-null  bool
 10  parking              15588 non-null  bool
 11  has_basement         15588 non-null  bool
 12  is_barrier-free      15588 non-null  bool
 13  has_built-in_kitchen 15588 non-null  bool
 14  has_bathtub          15588 non-null  bool
 15  has_shower           15588 non-null  bool
 16  flooring_type        15588 non-null  object
 17  energy_source        15588 non-null  object
 18  heating_type         15588 non-null  object
 19  property_condition   15588 non-null  object
 20  year_built           7662 non-null   float64
 21  zip_code             15588 non-null  int64
 22  district             15588 non-null  object
dtypes: bool(10), float64(5), int64(1), object(7)
memory usage: 1.7+ MB
```

[4]:
```
import squarify

floor_counts = (
    df["floor"]
    .value_counts()
    .sort_index()
    .reset_index()
)

floor_counts.columns = ["floor", "count"]

# Treemap dimensions
x, y, dx, dy = 0, 0, 100, 100

# Normalize counts
sizes = squarify.normalize_sizes(
    floor_counts["count"],
    dx,
    dy
)

# Calculate rectangle coordinates
rects = squarify.squarify(
    sizes,
    x,
    y,
    dx,
    dy
)

# Convert to DataFrame
df_rects = pd.DataFrame(rects)

# Calculate ending coordinates
df_rects["x2"] = df_rects["x"] + df_rects["dx"]
df_rects["y2"] = df_rects["y"] + df_rects["dy"]

# Calculate the center of each rectangle
df_rects["x_center"] = df_rects["x"] + df_rects["dx"] / 2
df_rects["y_center"] = df_rects["y"] + df_rects["dy"] / 2

# Combine everything
df_treemap = pd.concat(
    [
        floor_counts.reset_index(drop=True),
        df_rects.reset_index(drop=True)
    ],
    axis=1
)

rectangles = (
    alt.Chart(df_treemap)
    .mark_rect(
        stroke="white",
        strokeWidth=2
    )
    .encode(
        x=alt.X(
            "x:Q",
            scale=alt.Scale(domain=[0, 100]),
            axis=None
        ),
        y=alt.Y(
            "y:Q",
            scale=alt.Scale(domain=[0, 100]),
            axis=None
        ),
        x2="x2:Q",
        y2="y2:Q",

        color=alt.Color(
            "floor:O",
            legend=alt.Legend(title="Number of the floor")
        ),

        tooltip=[
            alt.Tooltip(
                "floor:O",
                title="Floors"
            ),
            alt.Tooltip(
                "count:Q",
                title="Number of properties"
            )
        ]
    )
)

labels = (
    alt.Chart(df_treemap)
    .mark_text(
        align="center",
        baseline="middle",
        fontSize=20,
        fontWeight="bold",
        color="white"
    )
    .encode(
        x=alt.X(
            "x_center:Q",
            scale=alt.Scale(domain=[0, 100]),
            axis=None
        ),
        y=alt.Y(
            "y_center:Q",
            scale=alt.Scale(domain=[0, 100]),
            axis=None
        ),
        text=alt.Text("floor:O")
    )
)

chart = (
    (rectangles + labels)
    .properties(
        width=500,
        height=500,
        title="Distribution of Number of the Floors"
    )
)

chart.display()
```



The treemap displays the distribution of the number of the floors. Most of the apartments happen to be on 1st floor. 4th, 2nd and 3rd floors are also very popular. This kind of graph is very good for comparing groups, for example, we notice that there are clearly more apartments on the 3rd floor than on the 6-11+ floors together. For more precise comparisons, we get the exact number of the floors on hover.

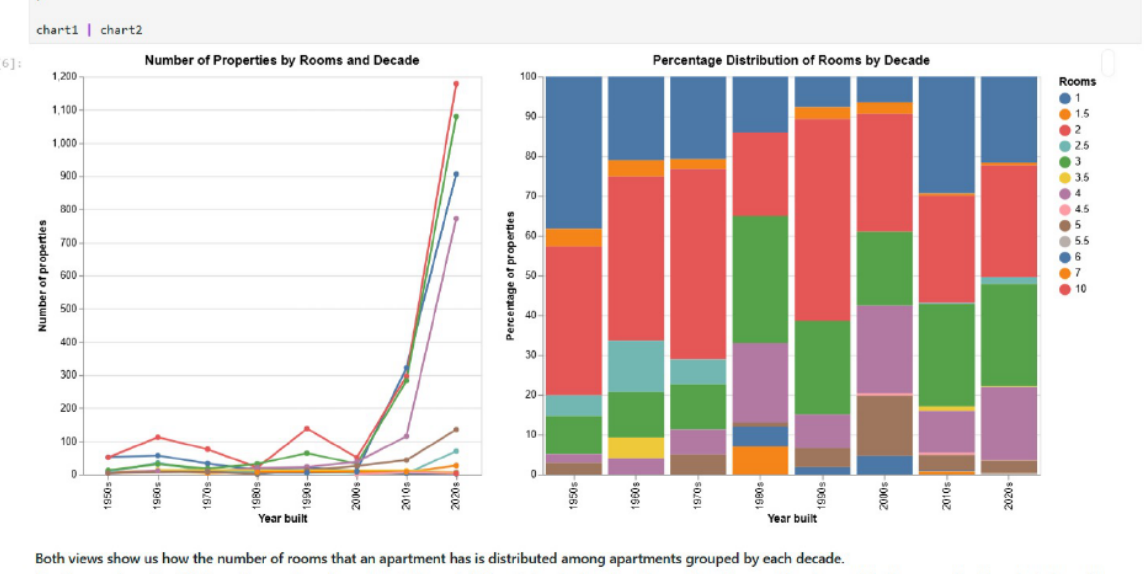
[5]:
```
# Sort by year built
df = df.sort_values("year_built")

# Count buildings for each year
yearly_count = (
    df.groupby("year_built")
    .size()
    .reset_index(name="count")
)

# Calculate cumulative count
yearly_count["cumulative_count"] = yearly_count["count"].cumsum()

zoom = alt.selection_interval(bind="scales", encodings=["x"])

chart = (
    alt.Chart(yearly_count)
    .mark_line(point=True)
    .encode(
        x=alt.X(
            "year_built:Q",
            title="Year Built"
        ),
        y=alt.Y(
            "cumulative_count:Q",
            title="Cumulative Number of Properties"
        ),
        tooltip=[
            alt.Tooltip("year_built:Q", title="Year Built"),
            alt.Tooltip("count:Q", title="Properties Built That Year"),
            alt.Tooltip(
                "cumulative_count:Q",
                title="Cumulative Properties"
            )
        ]
    )
    .add_params(zoom)
    .properties(
        width=700,
        height=400,
        title="Cumulative Properties Built Over Time"
    )
)

chart.display()
```



The line graph shows the number of apartments built until a certain year. Basically, in 1900, it shows how many apartments were built in 1900 and the years before. We notice that there were barely any apartments before 1900 and then we notice a spike in the year 1900. These could be caused by the renters putting rounded numbers instead of the exact year. Between 1900 and 1950 we notice a steady, slow growth, interrupted by the 2 World Wars. After the wars the growth seems steeper and it explodes after the year 2010.

[6]:
```
df_plot = df.dropna(subset=["year_built", "rooms"]).copy()

df_plot["year_built"] = pd.to_numeric(df_plot["year_built"], errors="coerce")
df_plot["rooms"] = pd.to_numeric(df_plot["rooms"], errors="coerce")
df_plot = df_plot.dropna(subset=["year_built", "rooms"])

df_plot["decade_start"] = (df_plot["year_built"] // 10) * 10
df_plot["decade"] = df_plot["decade_start"].astype(int).astype(str) + "s"

# Clean float formatting (1.5 stays "1.5", 2.0 becomes "2")
df_plot["rooms_formatted"] = df_plot["rooms"].apply(lambda x: f"{float(x):g}")

rooms_sort_order = (
    df_plot[["rooms", "rooms_formatted"]]
    .drop_duplicates()
    .sort_values("rooms")["rooms_formatted"]
    .tolist()
)

counts = (
    df_plot.groupby(["decade_start", "decade", "rooms", "rooms_formatted"])
    .size()
    .reset_index(name="count")
)
counts["percentage"] = (
    counts["count"] / counts.groupby("decade_start")["count"].transform("sum") * 100
)
counts = counts[counts["decade_start"] >= 1950].copy()

# Pivot to wide format so each room category is its own field -> lets vega-tooltip render one row per category by default
room_values = sorted(df_plot["rooms"].unique())

def safe_col(x):           # e.g. 1.5 -> "rooms_1_5"
    return "rooms_" + f"{float(x):g}".replace(".", "_")

def label(x):              # e.g. 1.5 -> "1.5"
    return f"{float(x):g}"

col_map = {v: safe_col(v) for v in room_values}

pivot = (
    counts.pivot_table(
        index=["decade_start", "decade"],
        columns="rooms",
        values="count",
        fill_value=0,
    )
    .reindex(columns=room_values, fill_value=0)
    .reset_index()
)
pivot.columns = [col_map.get(c, c) for c in pivot.columns]

tooltip_fields = [alt.Tooltip("decade:N", title="Decade")] + [
    alt.Tooltip(f"{col_map[v]}:Q", title=f"{label(v)} rooms")
    for v in room_values
]

hover = alt.selection_point(on="mouseover", nearest=True, fields=["decade"], empty=False)

lines = (
    alt.Chart(counts)
    .mark_line(point=True)
    .encode(
        x=alt.X("decade:O", sort=alt.SortField(field="decade_start", order="ascending"), title="Year built"),
        y=alt.Y("count:Q", title="Number of properties"),
        color=alt.Color("rooms_formatted:N", title="Rooms", sort=rooms_sort_order),
    )
)

hover_layer = (
    alt.Chart(pivot)
    .mark_point(opacity=0)
    .encode(
        x=alt.X("decade:O", sort=alt.SortField(field="decade_start", order="ascending")),
        tooltip=tooltip_fields,
    )
    .add_params(hover)
)

rule = (
    alt.Chart(counts)
    .mark_rule(color="gray")
    .encode(
        x="decade:O",
        opacity=alt.condition(hover, alt.value(0.5), alt.value(0)),
    )
)

highlight = (
    alt.Chart(counts)
    .mark_circle(size=100)
    .encode(
        x="decade:O",
        y="count:Q",
        color=alt.Color("rooms_formatted:N", title="Rooms", sort=rooms_sort_order),
        opacity=alt.condition(hover, alt.value(1), alt.value(0)),
    )
)

chart1 = (lines + rule + highlight + hover_layer).properties(
    width=400,
    height=400,
    title="Number of Properties by Rooms and Decade",
)

chart2 = (
    alt.Chart(counts)
    .mark_bar()
    .encode(
        x=alt.X("decade:O", sort=alt.SortField(field="decade_start", order="ascending"), title="Year built"),
        y=alt.Y("percentage:Q", title="Percentage of properties", scale=alt.Scale(domain=[0, 100])),
        color=alt.Color("rooms_formatted:N", title="Rooms", sort=rooms_sort_order),
        tooltip=[
            alt.Tooltip("decade:O", title="Decade"),
            alt.Tooltip("rooms_formatted:N", title="Rooms"),
            alt.Tooltip("count:Q", title="Properties"),
            alt.Tooltip("percentage:Q", title="Percentage", format=".1f"),
        ],
    )
    .properties(
        width=500,
        height=400,
        title="Percentage Distribution of Rooms by Decade",
    )
)

chart1 | chart2
```

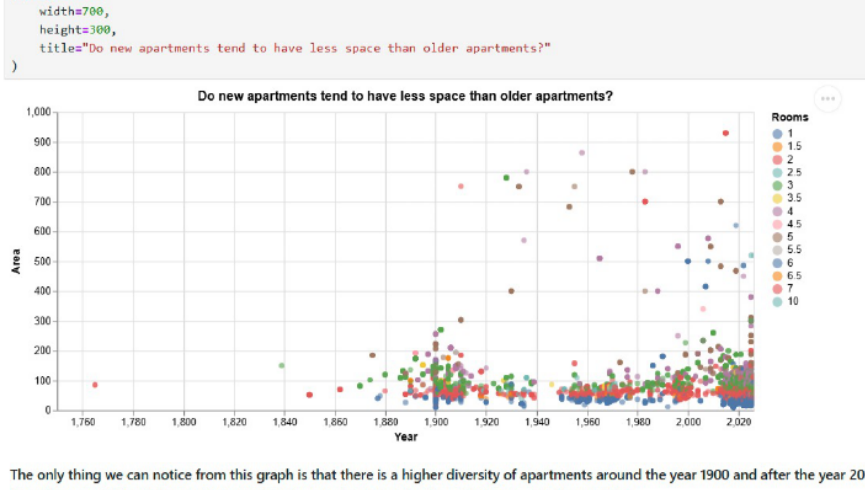
[6]:



Both views show us how the number of rooms that an apartment has is distributed among apartments grouped by each decade.
The line graph on the left shows the number of apartments through different decades based on the number of rooms they have, while the normalized bar chart shows it as a portion of a whole.
While there is not a clear trend throughout time, we can still notice some visible insights from both graphs. We notice that 2-room apartments are the most common ones. 1-room apartments seem to also be common, but during 1980-2000 there seem to be very few of them. We notice a relative rise in the number of apartments with many rooms, 3-room apartments seem to become very common, while 4, 5 and 6-room apartments seem to gain a larger portion of the market.

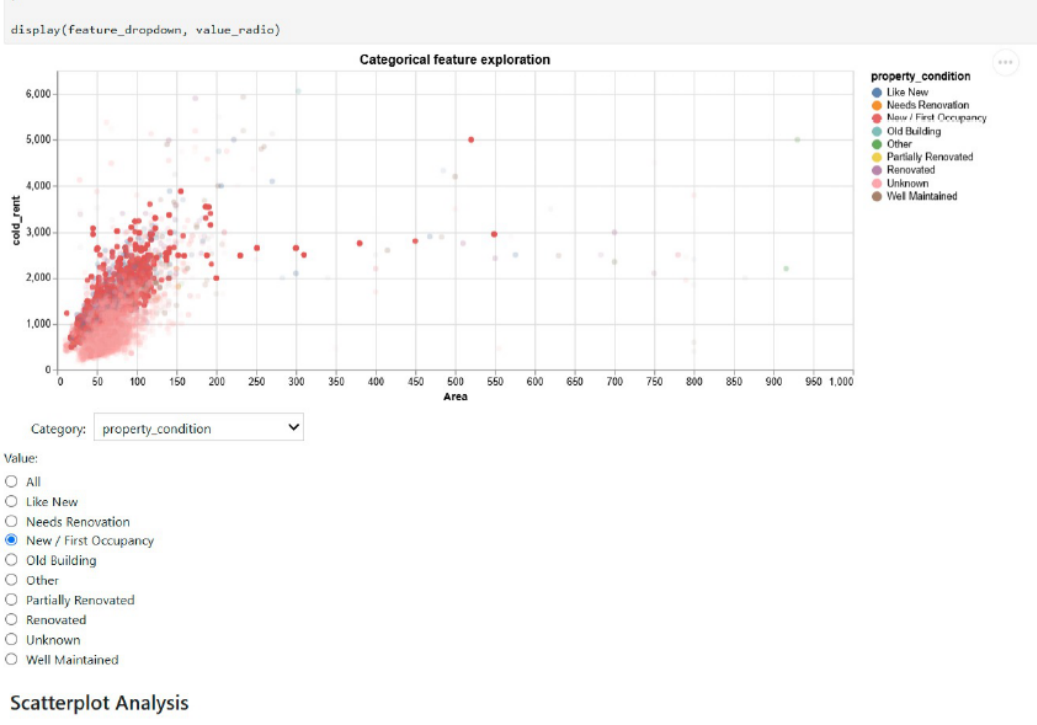
[7]:
```
zoom = alt.selection_interval(bind='scales', encodings=['x'])

alt.Chart(df).mark_point(filled=True).encode(
    alt.X('year_built:Q', title="Year", scale=alt.Scale(domain=[1750, 2026])),
    alt.Y('area:Q', title="Area", scale=alt.Scale(domain=[0, 1000])),
    alt.Color('rooms:N', legend=alt.Legend(title="Rooms")),
    opacity=alt.value(0.6),
    tooltip=['area:Q', 'year_built:Q', 'rooms:N']
).add_params(
    zoom
).properties(
    width=700,
    height=300,
    title="Do new apartments tend to have less space than older apartments?"
)
```

[7]:



The only thing we can notice from this graph is that there is a higher diversity of apartments around the year 1900 and after the year 2000, but we cannot tell for sure if there is a correlation between the year that the apartment was built and the amount of space it has.

[21]:
```
import ipywidgets as widgets
from IPython.display import display

category_options = [
    'has_balkon', 'has_garten', 'free_from', 'has_terrasse', 'elevator', 'parking',
    'has_basement', 'is_barrier-free', 'has_built-in_kitchen', 'has_bathtub',
    'has_shower', 'flooring_type', 'energy_source', 'heating_type', 'property_condition'
]

feature_dropdown = widgets.Dropdown(
    options=category_options,
    value='has_balkon',
    description='Category:'
)

value_radio = widgets.RadioButtons(
    options=['All'],
    value='All',
    description='Value:'
)


def get_value_options(feature):
    vals = sorted(
        df[feature].dropna().unique().tolist(),
        key=str
    )
    return ['All'] + vals


def make_chart(feature, value):

    cond = (
        alt.value(0.8)
        if value == 'All'
        else alt.condition(
            alt.FieldEqualPredicate(
                field=feature,
                equal=value
            ),
            alt.value(0.9),
            alt.value(0.05)
        )
    )

    zoom = alt.selection_interval(
        bind='scales',
        encodings=['x']
    )

    return (
        alt.Chart(df)
        .mark_point(filled=True)
        .encode(
            alt.X(
                'area:Q',
                title='Area',
                scale=alt.Scale(domain=[0, 1000])
            ),
            alt.Y(
                'cold_rent:Q',
                title='cold_rent'
            ),
            alt.Color(
                f'{feature}:N',
                legend=alt.Legend(title=feature)
            ),
            opacity=cond,
            tooltip=[
                'cold_rent:Q',
                'area:Q',
                'year_built:Q',
                'rooms:N',
                f'{feature}:N'
            ]
        )
        .add_params(zoom)
        .properties(
            width=800,
            height=300,
            title='Categorical feature exploration'
        )
    )


value_radio.options = get_value_options(feature_dropdown.value)


chart_handle = display(
    make_chart(
        feature_dropdown.value,
        value_radio.value
    ),
    display_id=True
)


def update_chart(*args):
    feature = feature_dropdown.value
    value = value_radio.value

    chart_handle.update(
        make_chart(feature, value)
    )


def on_feature_change(change):
    if change['name'] != 'value':
        return

    value_radio.unobserve(update_chart, names='value')

    value_radio.options = get_value_options(
        feature_dropdown.value
    )
    value_radio.value = 'All'

    value_radio.observe(update_chart, names='value')

    update_chart()

feature_dropdown.observe(
    on_feature_change,
    names='value'
)

value_radio.observe(
    update_chart,
    names='value'
)

display(feature_dropdown, value_radio)
```



Value:
○ All
○ Like New
○ Needs Renovation
● New / First Occupancy
○ Old Building
○ Other
○ Partially Renovated
○ Renovated
○ Unknown
○ Well Maintained

Scatterplot Analysis

The scatterplot above illustrates the relationship between **area** and **cold rent**, while allowing the clusters associated with different categorical features to be explored interactively. This makes it possible to examine how each feature is associated with rent prices by comparing the position and spread of its clusters. A feature appears to have a stronger association with rent when its categories form clearly separated clusters, while substantial overlap between categories suggests that the feature has little visible effect.

Features with a Clear Positive Association

[Free from] shows a relatively clear separation between the categories. The **"immediately"** category is positioned mainly in the upper-right area of the plot, indicating higher rents for apartments with similar areas. **"Not immediately"** appears more centrally, suggesting lower prices than "immediately" but generally higher prices than **"unknown."** The **"unknown"** category is very widely distributed and covers a large portion of the plot, including much of the area occupied by "not immediately." This overlap could indicate that the unknown values contain properties that are similar to those classified as "not immediately," although this cannot be confirmed from the scatterplot alone.

[Has built-in kitchen] shows a clear positive association with rent. The cluster for properties with a built-in kitchen is shifted toward higher prices compared with properties without one, suggesting that this feature is strongly associated with more expensive apartments.

[Has shower] also shows a clear positive association. Properties with a shower tend to be positioned higher in the plot, indicating higher cold rents for comparable apartment sizes.

[Elevator] appears to be associated with higher prices, although the effect is relatively weak. The clusters overlap considerably, but properties with an elevator appear to be slightly shifted toward higher rents.

Features with Little or No Clear Association

[Has balcony] does not show a clear difference between the categories. The clusters overlap substantially, suggesting that having a balcony does not have a strong visible association with cold rent in this plot.

[Has garden] similarly shows no clear separation between categories. The distributions overlap considerably, making it difficult to identify a meaningful association with rent.

[Parking] also does not show a clear effect. The cluster for properties without parking covers almost the entire distribution of properties with parking, indicating substantial overlap between the two categories.

[Has basement] shows essentially no visible difference between categories. The clusters occupy very similar regions of the plot.

[Is barrier-free] also shows no strong association with rent. There may be a slight tendency for the **"true"** category to be positioned toward higher prices, but the difference is small and the clusters overlap considerably.

Features Showing an Unexpected or Opposite Association

[Has terrasse] produces an interesting result because the observed pattern goes against the expected relationship. Instead of properties with a terrace being associated with higher rents, the cluster appears to suggest the opposite: apartments **without** a terrace are positioned toward higher prices. This may be caused by other factors correlated with the presence or absence of a terrace, so the result should not be interpreted as evidence that terraces reduce rent.

[Has bathtub] shows a similarly unexpected pattern. The cluster for properties with a bathtub is positioned toward the lower-right portion of the plot, indicating lower rents for apartments with comparable areas. This is again the opposite of what might intuitively be expected and could reflect differences in other property characteristics rather than a direct negative relationship between bathtubs and rent.

Flooring Type

[Flooring type] produces a more difficult pattern to interpret because some categories have very different sample sizes.

"Carpet" does not show a clear effect, but there are very few observations in this category, so there is insufficient data to draw a strong conclusion.

"Unknown" and **"wood"** contain very large numbers of observations and therefore cover a substantial portion of the plot. Despite this overlap, **"wood"** appears to be somewhat more associated with higher prices.

"Tiles" does not show a clear association with rent. However, an interesting pattern is that tiles appear frequently among apartments with larger areas.

"Stone" appears to be associated with somewhat lower prices, while **"laminate"** is concentrated more strongly among smaller and cheaper apartments. This may indicate an association with lower-priced properties, although the flooring type itself may not be the underlying reason for the difference.

Property Characteristics and Condition

[Property condition] shows some of the strongest differences between categories. **"New (first occupancy)"** and **"renovated"** are strongly associated with higher prices and are concentrated toward the upper-right portion of the plot. **"Well maintained"** is also associated with relatively high rents and appears frequently among apartments with larger areas, although its cluster is somewhat more spread out.

"Partially renovated" is positioned closer to the middle of the distribution, although there are very few observations in this category, making it difficult to draw strong conclusions.

The categories **"old building," "other,"** and **"needs renovation"** contain relatively few observations and appear slightly below the average price range. Because of their small sample sizes, however, their apparent differences should be interpreted cautiously.

The **"unknown"** category is particularly notable. It forms a very large and widely distributed cluster and shows a strong association with lower prices. In fact, many of the observations at the very bottom of the plot belong to this category. This suggests that missing or unknown information about property condition may be concentrated among lower-priced listings.

Energy and Heating Features

[Energy source] shows substantial differences between categories. **"District heating"** is clearly associated with higher prices and appears frequently among the most expensive apartments. It represents one of the most visibly expensive clusters in the plot.

"Electric" is also associated with relatively high rents, although the cluster is generally positioned below district heating. **"Gas"** is associated with above-average prices as well, but appears to be somewhat lower than electric heating. It also occurs frequently among apartments with larger areas.

"Heat pump" and **"geothermal"** appear to occupy a similar price range to gas. **"Hybrid"** appears to have a distribution more similar to electric heating. In contrast, **"oil/coal"** and **"other"** appear to be associated with below-average prices.

The **"unknown"** energy-source category is very widely distributed across the plot. Nevertheless, it appears to be slightly more concentrated toward lower prices.

[Heating] also shows some visible differences. **"Underfloor heating"** is clearly associated with higher prices and is concentrated toward the upper portion of the plot. **"Individual heating"** also appears to be associated with above-average prices.

In contrast, **"central heating"** and **"unknown"** contain very large numbers of observations, resulting in substantial overlap across the plot. Therefore, no clear price effect can be identified for these categories from the scatterplot alone. It is also worth noting that central heating appears frequently among apartments with larger areas.

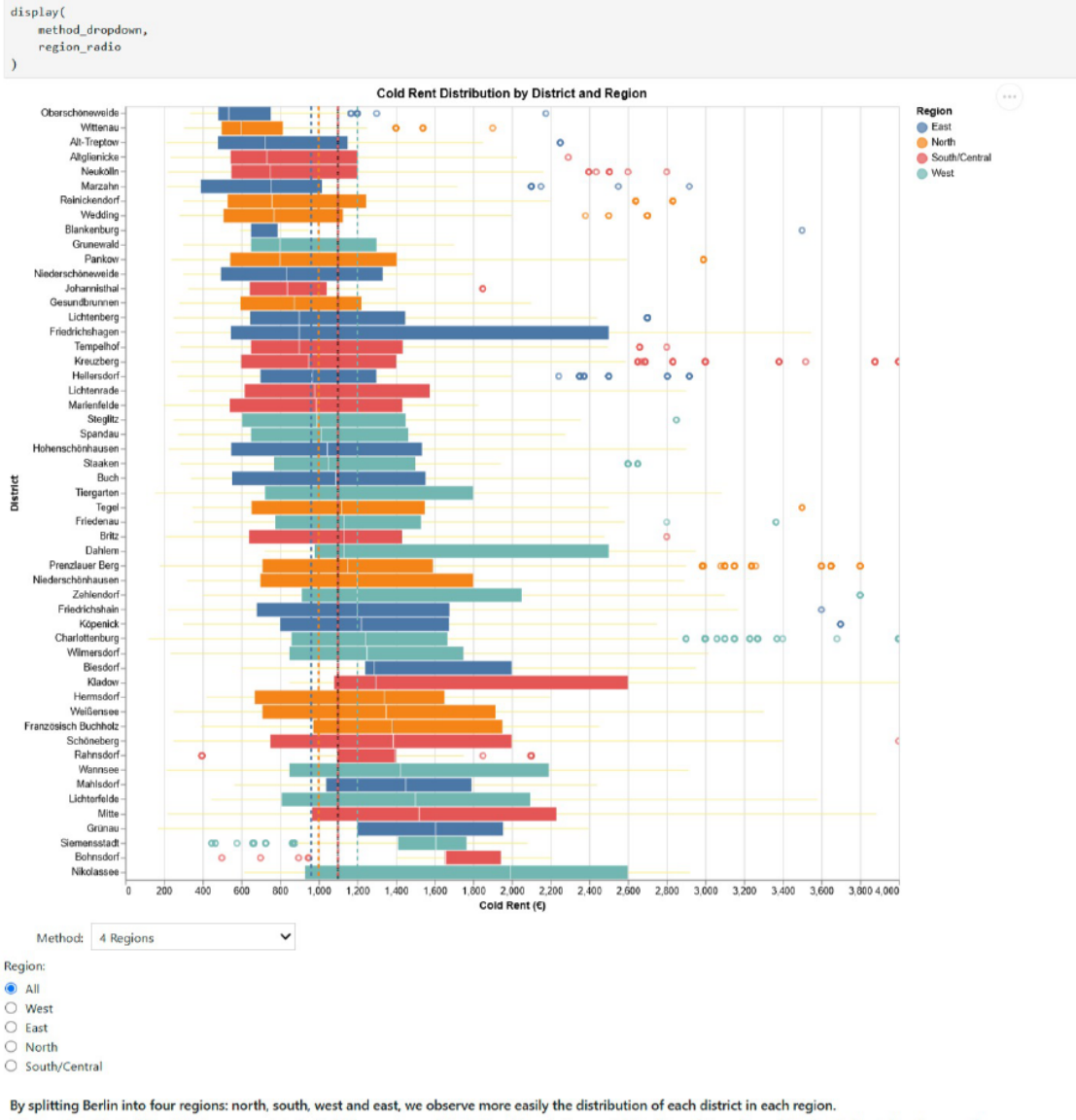
[9]:
```
west_berlin = ["Charlottenburg", "Wilmersdorf", "Spandau", "Zehlendorf", "Dahlem", "Grunewald", "Wannsee", "Steglitz",
               "Lichterfelde", "Friedenau", "Siemensstadt","Staaken","Nikolassee","Tiergarten"]
east_berlin = ["Friedrichshain", "Lichtenberg", "Hohenschönhausen", "Marzahn", "Hellersdorf", "Köpenick", "Friedrichshagen",
               "Oberschöneweide", "Niederschöneweide", "Alt-Treptow", "Grünau", "Mahlsdorf", "Biesdorf", "Blankenburg", "Buch"]
north_berlin = ["Pankow", "Weißensee", "Niederschönhausen", "Hermsdorf", "Tegel", "Wittenau", "Reinickendorf", "Gesundbrunnen",
               "Wedding","Prenzlauer Berg","Französisch Buchholz"]
south_central_berlin = ["Mitte", "Neukölln", "Kreuzberg","Schöneberg", "Tempelhof", "Britz", "Lichtenrade", "Mariendorf",
               "Johannisthal", "Altglienicke", "Rahnsdorf", "Kladow", "Bohnsdorf"]

region_4 = {
    "West": west_berlin,
    "East": east_berlin,
    "North": north_berlin,
    "South/Central": south_central_berlin
}
```

[10]:
```
west = [
    "Charlottenburg", "Wilmersdorf", "Spandau", "Zehlendorf", "Dahlem",
    "Grunewald", "Wannsee", "Steglitz", "Lichterfelde", "Friedenau", "Siemensstadt",
    "Hermsdorf", "Tegel", "Wittenau", "Reinickendorf",
    "Neukölln", "Kreuzberg", "Schöneberg", "Tempelhof", "Britz",
    "Lichtenrade", "Mariendorf", "Kladow", "Tiergarten", "Wedding",
    "Prenzlauer Berg", "Staaken", "Nikolassee", "Französisch Buchholz", "Bohnsdorf"
]

east = [
    "Friedrichshain", "Lichtenberg", "Hohenschönhausen", "Marzahn", "Hellersdorf",
    "Köpenick", "Friedrichshagen", "Oberschöneweide", "Niederschöneweide",
    "Alt-Treptow", "Grünau", "Mahlsdorf", "Biesdorf", "Blankenburg", "Buch",
    "Pankow", "Weißensee", "Niederschönhausen", "Gesundbrunnen",
    "Mitte", "Johannisthal", "Altglienicke", "Rahnsdorf"
]

region_2 = {
    "West": west,
    "East": east,
}
```

[11]:
```
center = [
    "Mitte",
    "Kreuzberg",
    "Friedrichshain",
    "Schöneberg",
    "Gesundbrunnen",
    "Tiergarten",
    "Prenzlauer Berg",
    "Wedding",
]
surrounding = [
    "Charlottenburg", "Wilmersdorf",
    "Pankow", "Weißensee", "Niederschönhausen",
    "Tempelhof", "Steglitz", "Friedenau",
    "Lichtenberg", "Alt-Treptow", "Neukölln",
    "Johannisthal", "Oberschöneweide", "Niederschöneweide", "Staaken",
    "Nikolassee","Bohnsdorf","Französisch Buchholz"
]
suburbs = [
    "Spandau", "Zehlendorf", "Dahlem", "Grunewald", "Wannsee",
    "Lichterfelde", "Siemensstadt",
    "Hermsdorf", "Tegel", "Wittenau", "Reinickendorf",
    "Britz", "Lichtenrade", "Mariendorf", "Kladow",
    "Hohenschönhausen", "Marzahn", "Hellersdorf",
    "Köpenick", "Friedrichshagen", "Grünau",
    "Mahlsdorf", "Biesdorf", "Blankenburg", "Buch",
    "Altglienicke", "Rahnsdorf"
]
region_3 = {
    "Center": center,
    "Surrounding": surrounding,
    "Suburbs": suburbs
}
```

[12]:
```
region_maps = {
    "region_4": region_4,
    "region_2": region_2,
    "region_3": region_3
}


def add_region_column(data, method):
    mapping = region_maps[method]

    district_to_region = {
        district: region
        for region, districts in mapping.items()
        for district in districts
    }

    result = data.copy()
    result[method] = result["district"].map(district_to_region)

    return result
```

[13]:
```
method_dropdown = widgets.Dropdown(
    options=[
        ("4 Regions", "region_4"),
        ("2 Regions (East/West)", "region_2"),
        ("Center/Surrounding/Suburbs", "region_3"),
    ],
    value="region_4",
    description="Method:"
)
region_radio = widgets.RadioButtons(options=["All"], value="All", description="Region:")
output = widgets.Output()

overall_median = df["cold_rent"].median()

def get_region_options(method):
    return ["All"] + list(region_maps[method].keys())

def make_chart(method, region):

    subset = add_region_column(df.dropna(subset=["district", "cold_rent"]), method)
    subset = subset.dropna(subset=[method])

    if region != "All":
        subset = subset[subset[method] == region]

    median_order = (
        subset.groupby("district")["cold_rent"]
        .median()
        .sort_values()
        .index
        .tolist()
    )

    box = alt.Chart(subset).mark_boxplot(
        color="#f7ec8b",
        clip=True
    ).encode(
        x=alt.X(
            "cold_rent:Q",
            title="Cold Rent (€)",
            scale=alt.Scale(domain=[0, 4000])
        ),
        y=alt.Y(
            "district:N",
            title="District",
            sort=median_order
        ),
        color=alt.Color(
            f"{method}:N",
            legend=alt.Legend(title="Region")
        ),
        tooltip=[
            "district:N",
            f"{method}:N",
            "cold_rent:Q"
        ]
    )

    overall_line = (
        alt.Chart(pd.DataFrame({"x": [overall_median]}))
        .mark_rule(
            color="black",
            strokeDash=[1],
            size=2
        )
        .encode(
            x="x:Q"
        )
    )

    if region == "All":

        region_medians = (
            subset.groupby(method)["cold_rent"]
            .median()
            .reset_index()
        )

        region_medians.columns = ["region", "region_median"]

        region_line = (
            alt.Chart(region_medians)
            .mark_rule(
                strokeDash=[4, 4],
                size=2
            )
            .encode(
                x="region_median:Q",
                color=alt.Color(
                    "region:N",
                    legend=None
                )
            )
        )

    else:

        region_median_val = subset["cold_rent"].median()

        region_line = (
            alt.Chart(
                pd.DataFrame({"x": [region_median_val]})
            )
            .mark_rule(
                color="blue",
                strokeDash=[4, 4],
                size=2
            )
            .encode(
                x="x:Q"
            )
        )

    return (
        box + overall_line + region_line
    ).properties(
        width=800,
        height=800,
        title="Cold Rent Distribution by District and Region"
    )

region_radio.options = get_region_options(method_dropdown.value)
region_radio.value = "All"

chart_handle = display(
    make_chart(
        method_dropdown.value,
        region_radio.value
    ),
    display_id=True
)


def update_chart(*args):
    chart_handle.update(
        make_chart(
            method_dropdown.value,
            region_radio.value
        )
    )


def on_method_change(change):

    if change["name"] != "value":
        return

    region_radio.unobserve(
        update_chart,
        names="value"
    )

    region_radio.options = get_region_options(
        method_dropdown.value
    )

    region_radio.value = "All"

    region_radio.observe(
        update_chart,
        names="value"
    )

    update_chart()

method_dropdown.observe(
    on_method_change,
    names="value"
)

region_radio.observe(
    update_chart,
    names="value"
)

display(
    method_dropdown,
    region_radio
)
```



Region:
● All
○ West
○ East
○ North
○ South/Central

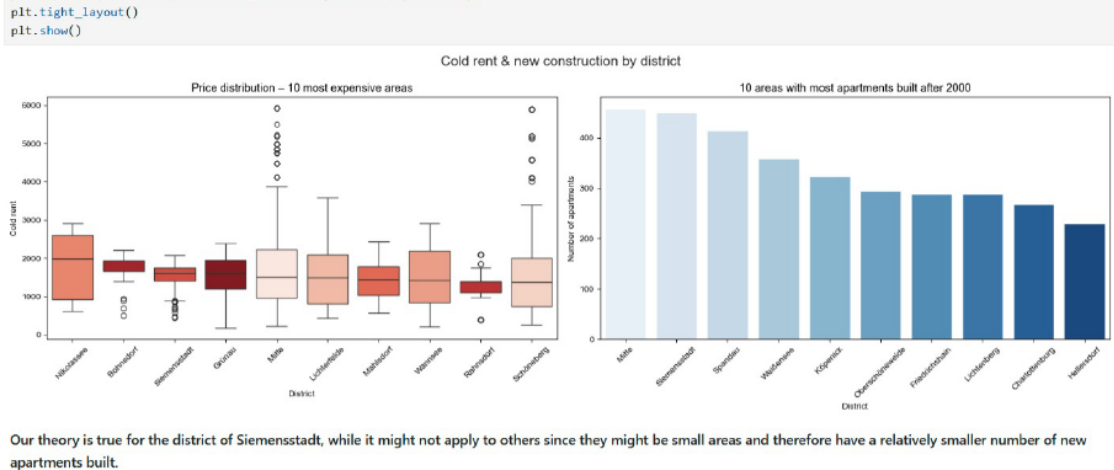
By splitting Berlin into four regions: north, south, west and east, we observe more easily the distribution of each district in each region.
West region: Most of the districts are associated with higher rent overall in Berlin, with only Grunewald, Steglitz and Spandau and Staaken being below the overall average, while Nikolassee leads the western part, followed by Siemensstadt, Lichterfelde and Wannsee. This region has a much higher median compared to the overall median.
East region: In this region most of the districts tend to have a lower distribution of prices compared to the average, with only 5 regions having a higher median than the global median. The districts of Grünau and Mahlsdorf have a very high distribution of prices, while Friedrichshagen offers a much wider distribution compared to the rest. This region, in contradiction to the west region, has a much lower median compared to the overall median.
North region: It's 3 of the most expensive districts, Französisch Buchholz, Weißensee and Hermsdorf, don't make it even in the top 10 most expensive districts, with Französisch Buchholz being associated with the highest prices and Wittenau being associated with the lowest. Despite that, overall this area is associated with higher prices compared to the east region.
South region: Compared to the other 3 regions, the south has a more stable distribution of prices, with 5 districts having a relatively higher distribution. One could argue that Mitte should not be part of the south and by excluding that and Bohnsdorf we see almost a somewhat uniform distribution of prices in this region. This region has a very similar median to the overall median.

West Berlin vs East Berlin
When comparing the statistics of former East Berlin and West Berlin, there is barely any difference. When it comes to distribution, the West has a slightly higher distribution and a slightly higher average. Districts like Grünau and Mitte, which are very expensive, bring up the overall statistics of the eastern part.

Splitting by distance by city center:
Here the results are very meaningful economically, with areas near the city center being associated with a higher distribution of rent prices compared to other parts. The difference between overall distribution and average is huge. While surrounding districts have a visibly lower price distribution than the overall distribution, the same thing doesn't apply to suburban areas, whose distribution is actually slightly larger. This could be due to the fact that it contains a few districts with a very high price distribution like Siemensstadt, Grünau, Lichterfelde, Mahlsdorf, Wannsee, Rahnsdorf, Hermsdorf and Kladow.

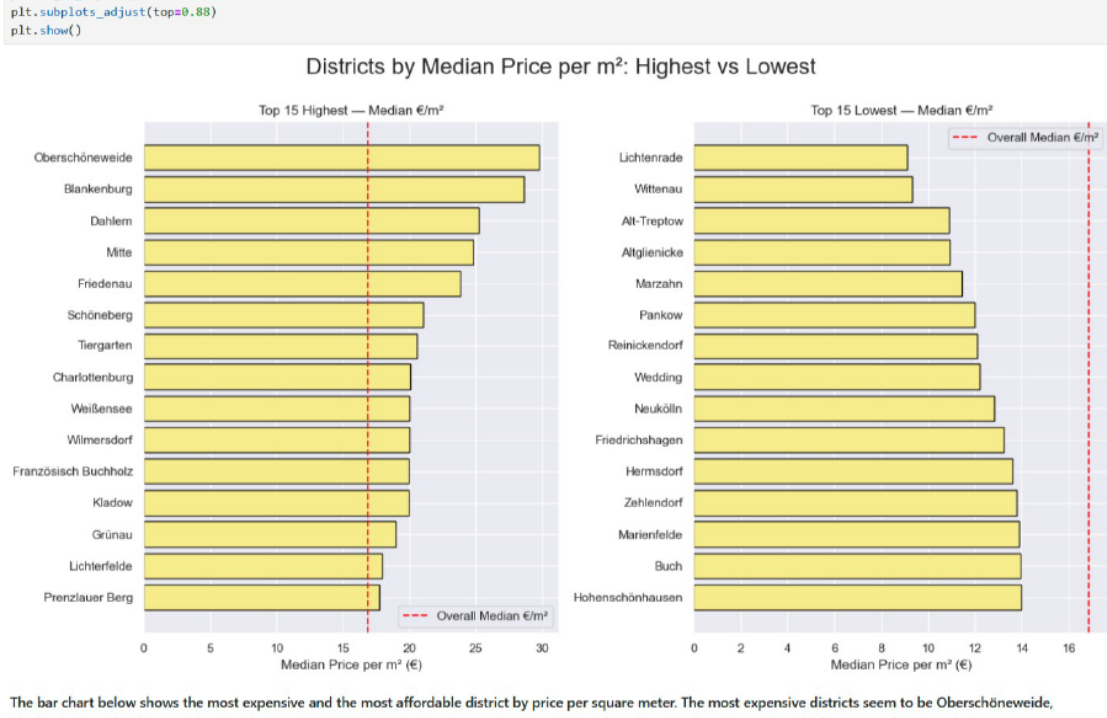
How is it possible that these areas very far from the city center have such high prices?
This could be due to the fact that these are very new constructions in these areas and new constructions tend to have better conditions and therefore, higher rent prices.
Let's test whether this is true or not.

[14]:
```
fig, axes = plt.subplots(1, 2, figsize=(18, 6))
sns.set(style="darkgrid")

# 10 most expensive areas by median price
top_expensive = (
    df.groupby("district")["cold_rent"]
    .median()
    .nlargest(10)
    .index
)

# 10 areas with most apartments built after 2000
new_counts = (
    df[df["year_built"] > 2000]
    .groupby("district")
    .size()
    .nlargest(10)
    .reset_index(name="count")
)

sns.boxplot(
    data=df[df["district"].isin(top_expensive)],
    x="district", y="cold_rent",
    hue="district", legend=False,
    order=top_expensive,
    ax=axes[0], palette="Reds"
)
axes[0].set_title("Price distribution - 10 most expensive areas", size=13)
axes[0].set_xlabel("District")
axes[0].set_ylabel("Cold rent")
axes[0].tick_params(axis='x', rotation=45)

sns.barplot(
    data=new_counts,
    x="district", y="count",
    hue="district", legend=False,
    order=new_counts["district"],
    ax=axes[1], palette="Blues"
)
axes[1].set_title("10 areas with most apartments built after 2000", size=13)
axes[1].set_xlabel("District")
axes[1].set_ylabel("Number of apartments")
axes[1].tick_params(axis='x', rotation=45)

plt.suptitle("Cold rent & new construction by district", size=16)
plt.tight_layout()
plt.show()
```



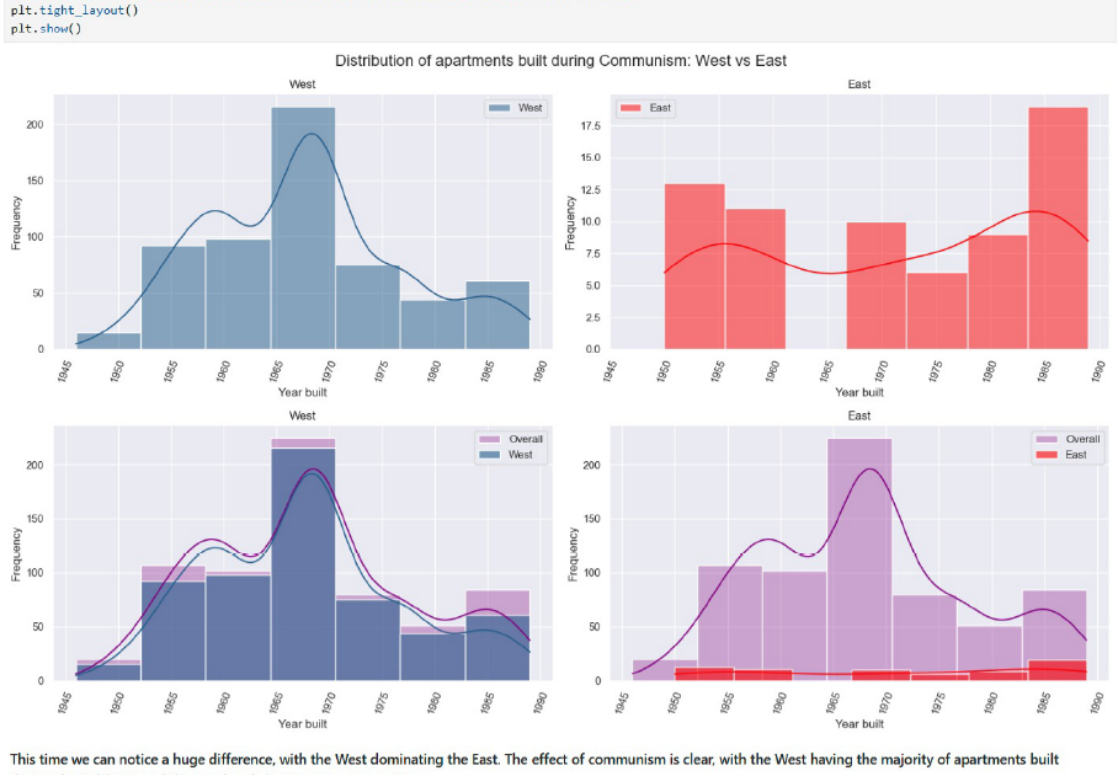
Our theory is true for the district of Siemensstadt, while it might not apply to others since they might be small areas and therefore have a relatively smaller number of new apartments built.

[15]:
```
df["price_per_m2"] = df["cold_rent"] / df["area"]

# Median price/m² per district
district_median = (
    df.groupby("district")["price_per_m2"]
    .median()
    .sort_values(ascending=False)
)

top15 = district_median.head(15)
bottom15 = district_median.tail(15).sort_values(ascending=True)

overall_median = df["price_per_m2"].median()

fig, axes = plt.subplots(1, 2, figsize=(14, 8))

# Left: Top 15 Highest
axes[0].barh(top15.index[::-1], top15.values[::-1], color="#f7ec8b", edgecolor="black")
axes[0].axvline(overall_median, linestyle="--", color="red", label="Overall Median €/m²")
axes[0].set_xlabel("Median Price per m² (€)")
axes[0].set_title("Top 15 Highest — Median €/m²")
axes[0].legend()

# Right: Top 15 Lowest
axes[1].barh(bottom15.index[::-1], bottom15.values[::-1], color="#f7ec8b", edgecolor="black")
axes[1].axvline(overall_median, linestyle="--", color="red", label="Overall Median €/m²")
axes[1].set_xlabel("Median Price per m² (€)")
axes[1].set_title("Top 15 Lowest — Median €/m²")
axes[1].legend()

fig.suptitle("Districts by Median Price per m²: Highest vs Lowest", size=20)
plt.tight_layout()
plt.subplots_adjust(top=0.88)
plt.show()
```



The bar chart below shows the most expensive and the most affordable district by price per square meter. The most expensive districts seem to be Oberschöneweide, Blankenburg and Dahlem, with prices between 25 and 30 euros per square meter, way higher than the overall median price, which seems to be around 16 euros per square meter. This is quite surprisingly, considering that these are non-central districts with very low distributions of prices but very high prices per square meter, indicating that the apartments in these areas might be relatively much smaller. The argument for a small sample size for these categories would be invalid, especially for Oberschöneweide, which has a considerable number of samples.
After the top 3, the leading areas are much expected, central expensive areas like Mitte, Friedenau, Schöneberg, Tiergarten and Charlottenburg also make it to the top of the list.
We also notice that the overall median is very close to the estimated price per square meter of 15.23 euros.
The most affordable areas, when referring to the price per square meter, seem to be Lichtenrade, positioned at a very remote corner of Berlin, Wittenau which is positioned very north, far from good districts and surprisingly Alt-Treptow, which seems to be in a very good location.

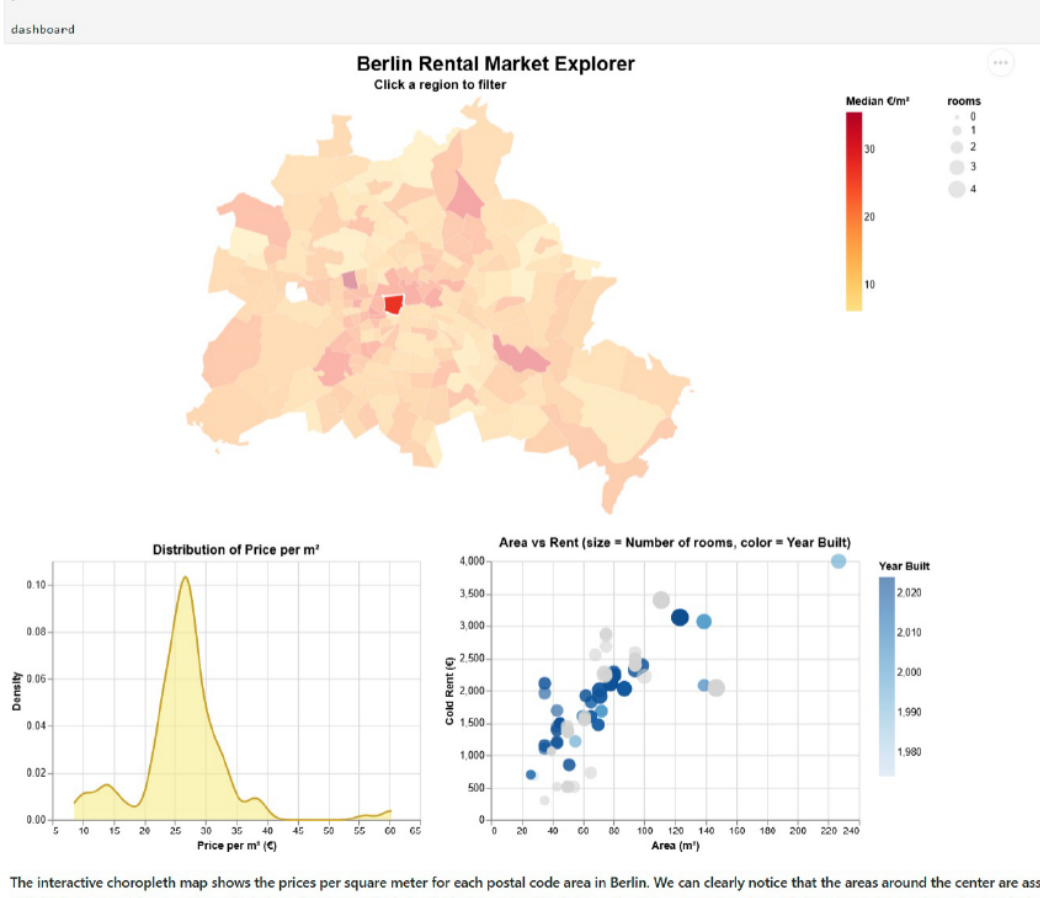
[16]:
```
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
sns.set(style="darkgrid")

ticks = range(1945, 1991, 5)
overall = df[(df["year_built"] >= 1945) & (df["year_built"] < 1990)]

region_colors = {
    "West": "#245e8c",
    "East": "red",
}

for i, region in enumerate(region_2.keys()):
    subset = df[df["district"].isin(region_2[region])]
    filtered = subset[(subset["year_built"] >= 1945) & (subset["year_built"] < 1990)]
    color = region_colors.get(region, "steelblue")  # fallback color

    # Row 1: region only
    sns.histplot(filtered["year_built"], bins=7, kde=True, ax=axes[0, i],
                 color=color, label=region)
    axes[0, i].set_xticks(ticks)
    axes[0, i].set_title(region)
    axes[0, i].set_xlabel('Year built')
    axes[0, i].set_ylabel('Frequency')
    axes[0, i].tick_params(axis='x', rotation=70)
    axes[0, i].legend()

    # Row 2: region + overall
    sns.histplot(overall["year_built"], bins=7, kde=True, ax=axes[1, i],
                 color="purple", alpha=0.3, label="Overall")
    sns.histplot(filtered["year_built"], bins=7, kde=True, ax=axes[1, i],
                 color=color, alpha=0.6, label=region)
    axes[1, i].set_xticks(ticks)
    axes[1, i].set_title(region)
    axes[1, i].set_xlabel('Year built')
    axes[1, i].set_ylabel('Frequency')
    axes[1, i].tick_params(axis='x', rotation=70)
    axes[1, i].legend()

plt.suptitle("Distribution of apartments built during Communism: West vs East", size=16)
plt.tight_layout()
plt.show()
```



This time we can notice a huge difference, with the West dominating the East. The effect of communism is clear, with the West having the majority of apartments built during the Cold War and the East barely having any constructions.

[17]:
```
import geopandas as gpd
import requests, zipfile, io

# Load PLZ boundaries
url = "https://raw.githubusercontent.com/funkeinteraktiv/Berlin-Geodaten/master/berlin_postleitzahlen.zip"
r = requests.get(url)
z = zipfile.ZipFile(io.BytesIO(r.content))
z.extractall("berlin_plz_shp")
gdf = gpd.read_file("berlin_plz_shp/berlin_postleitzahlen.shp")
gdf["PLZ99"] = gdf["PLZ99"].astype(str)

# Adjust "zip_code" to the actual column name
df["PLZ99"] = df["zip_code"].astype(str)

zip_stats = df.groupby("PLZ99")["price_per_m2"].median().reset_index()
merged = gdf.merge(zip_stats, on="PLZ99", how="left")

click = alt.selection_point(name="region_click", fields=["PLZ99"], on="click", empty="all")

map_chart = (
    alt.Chart(merged)
    .mark_geoshape(stroke="white")
    .encode(
        color=alt.Color("price_per_m2:Q", scale=alt.Scale(scheme="yelloworangered"), title="Median €/m²"),
        opacity=alt.condition(click, alt.value(1), alt.value(0.4)),
        strokeWidth=alt.condition(click, alt.value(2), alt.value(0.3)),
        tooltip=["PLZ99:N", "price_per_m2:Q"]
    )
    .add_params(click)
    .project(type="mercator")
    .properties(width=700, height=420, title="Click a region to filter")
)

kde = (
    alt.Chart(df)
    .transform_filter(click)
    .transform_density("price_per_m2", as_=["price_per_m2", "density"])
    .mark_area(opacity=0.6, color="#f7ec8b", line={"color": "#c9a227"})
    .encode(
        x=alt.X("price_per_m2:Q", title="Price per m² (€)"),
        y=alt.Y("density:Q", title="Density")
    )
    .properties(width=370, height=260, title="Distribution of Price per m²")
)

scatter = (
    alt.Chart(df)
    .transform_filter(click)
    .mark_circle(opacity=0.6, invalid=None)
    .encode(
        x=alt.X("area:Q", title="Area (m²)"),
        y=alt.Y("cold_rent:Q", title="Cold Rent (€)"),
        size=alt.Size("rooms:Q", title="rooms", scale=alt.Scale(range=[20, 300])),
        color=alt.condition(
            "datum.year_built === null",
            alt.value("lightgray"),
            alt.Color("year_built:Q", title="Year Built", scale=alt.Scale(scheme="blues"))
        ),
        tooltip=["area:Q", "cold_rent:Q", "rooms:Q", "year_built:Q", "PLZ99:N"]
    )
    .properties(width=370, height=260, title="Area vs Rent (size = Number of rooms, color = Year Built)")
)

dashboard = alt.vconcat(
    map_chart,
    alt.hconcat(kde, scatter, spacing=20)
).resolve_scale(color="independent").properties(
    title=alt.TitleParams(
        text="Berlin Rental Market Explorer",
        anchor="middle",
        fontSize=20
    )
)

dashboard
```

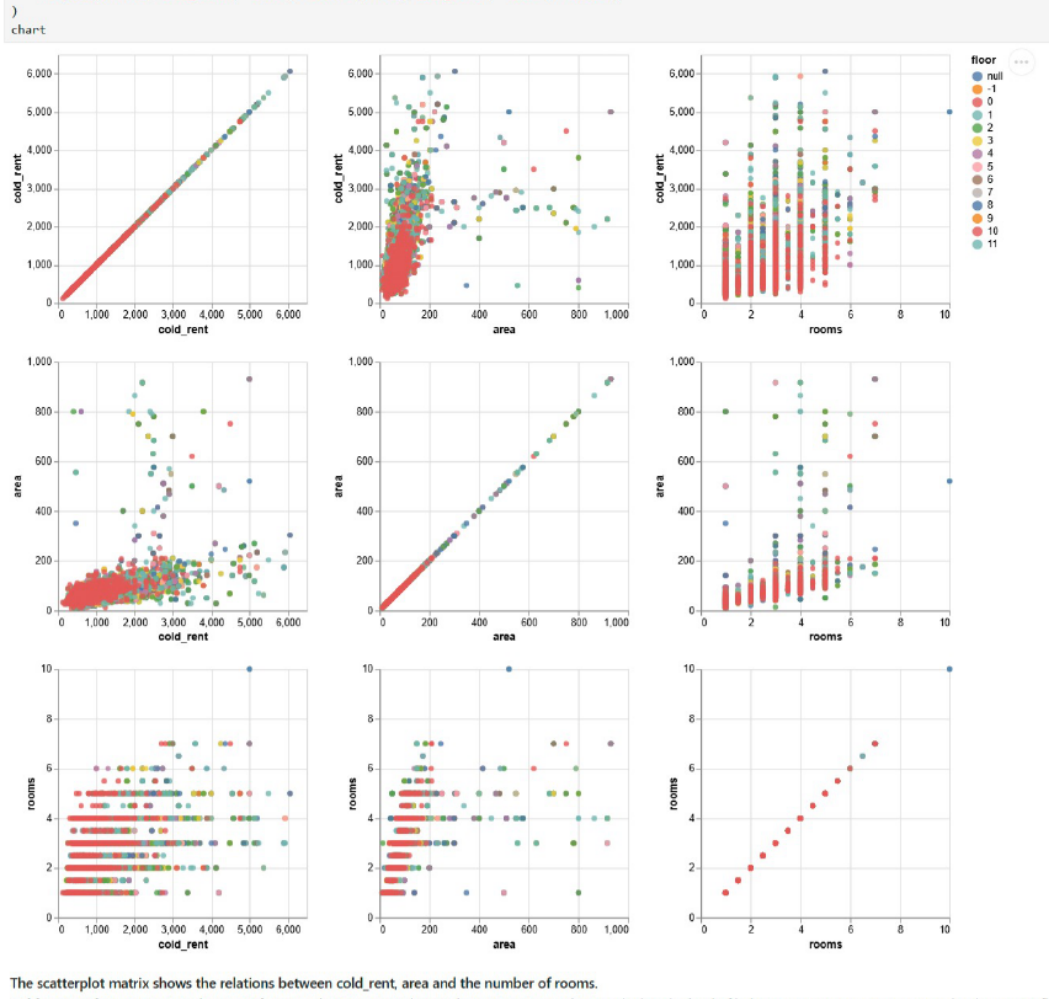
[17]:


Berlin Rental Market Explorer
Click a region to filter
Median €/m²
rooms
Distribution of Price per m²
Density
Price per m² (€)
Area vs Rent (size = Number of rooms, color = Year Built)
Cold Rent (€)
Area (m²)
Year Built

The interactive choropleth map shows the prices per square meter for each postal code area in Berlin. We can clearly notice that the areas around the center are associated with higher prices. There are 2 graphs below: the one on the left, which shows the distribution of price per square meter and the scatter plot on the right, which shows the relation of area and rent prices, with size encoded as the number of rooms, indicating higher size for more rooms, while color saturation (how strong the color is) is used to encode the year when the apartment was built, with higher saturation indicating newer apartments.
On demand the user can click on each region to see the distribution and scatterplot corresponding to that specific region.

[18]:
```
brush = alt.selection_interval(resolve="global")
click = alt.selection_point(fields=["floor"], on="click", empty="all")

chart = (
    alt.Chart(df)
    .mark_circle()
    .add_params(brush, click)
    .encode(
        alt.X(alt.repeat("column"), type="quantitative"),
        alt.Y(alt.repeat("row"), type="quantitative"),
        color=alt.condition(click, alt.Color("floor:N", title="Floor"), alt.value("grey")),
        opacity=alt.condition(brush, alt.value(0.8), alt.value(0.05)),
        tooltip=[
            alt.Tooltip("rooms:Q", title="Rooms"),
            alt.Tooltip("area:Q", title="Area"),
            alt.Tooltip("cold_rent:Q", title="Cold rent"),
            alt.Tooltip("floor:N", title="Floor")
        ]
    )
    .properties(width=250, height=250)
    .repeat(column=["cold_rent", "area", "rooms"], row=["cold_rent", "area", "rooms"])
)
chart
```

[18]:



The scatterplot matrix shows the relations between cold_rent, area and the number of rooms.
Cold rent and area: Between these two features, there is a very clear and strong positive relation, which is the kind of behavior we expect. Area seems to be the most influential variable.
Cold rent and number of rooms: Just like the area, there seems to be a clear, strong positive relation, but just not as strong as the area.
Area and number of rooms: Also for this case, there is a clear positive relation, with larger apartments having more rooms.
The apartment's floor and the rest of the features: When observing the correlation of the features with themselves separated by the floor number using color hue, we notice a relation between the floor number with price and apartment size. More expensive apartments are associated with apartments in higher floors. The same observation is to be noticed with area.